

Propuesta de mejora — Módulo "Bitácora" (agenda, tareas, gestiones y notas por expediente)

Documento de propuesta · v3 · 2026-07-05 · Estado: **borrador para evaluación** (no aprobado, no en desarrollo) Referencia de inspiración funcional: manual Lex-Doctor 11 (Agenda §XIII, Gestiones §XIV) + UX de Google Calendar / Google Tasks. Cambios v2: se reemplaza el "historial de procuraciones en la nube" por el modelo **Expediente seguido** (ficha por caso + historial acotado a hasta 2 procuraciones y hasta 2 informes); captura confirmada por deep-link (opción A); avisos solo in-app (sin email); píldora "Establecer como principal" en Mi Plan / Bitácora; gating por plan; integración Google descartada por ahora. Cambios v3: **selección múltiple** en la tabla de los visores (captura en lote) + link  a la ficha de casos ya seguidos; **exportación** (Excel legible + JSON restaurable, global y por caso) e **importación/restauración** desde backup con modos reemplazar/combinar, vista previa dry-run y respaldo automático previo. Cambios v4: alta **manual** de expedientes explicitada; ficha = vista integral del caso (datos + bitácora + historial) vs. sección Bitácora = vista temporal (calendario); **personalización de la ficha** (orden de secciones, registros visibles, modal "ver todos"); deep-links con **pestaña única reutilizada** (**target** fijo), sesión en uso sin re-login y botón Volver coherente vía History API. Cambios v5: sección de **autosuficiencia y fuentes**; **riesgo y complejidad explicados sin tecnicismos** (qué se toca en web y app, qué no se toca, reversibilidad); **estimación de costos** con números reales del servidor; alcance del historial precisado (**2+2 POR CASO**, alimentado por cada corrida); límite del querystring explicado y regla de recorte; **historial con selector última/anteúltima + modal**; botones "  Guardar procuración/informe" en visores (selección múltiple y modal del caso); **filtros y agrupación** en la vista global; **campos por tipo de entrada** orientados a la práctica jurídica (ref. Lex-Doctor + Google); vencimientos visibles en la ficha; edición global y por caso con acciones masivas; sección de **preguntas abiertas**. Cambios v6: **se desacopla "guardar ficha básica" de "guardar snapshot de procuración/informe"** (dos acciones independientes en vez de una combinada); **se reemplaza el mecanismo de transporte** de GET-por-querystring a **POST por formulario oculto autoenviado** (misma pestaña con nombre fijo, sin CORS, pero sin el límite práctico de 2.000 caracteres —elimina la restricción de "hasta 10 casos sin movimientos" del modo lote); **el botón de Bitácora se muda del sidebar al topbar** (una sola aparición, en la misma barra que los tabs Procurar/Informe/Monitor/Descargas y el botón de cerrar ventana); **se agrega el paso del tour de onboarding** que faltaba para explicar el botón nuevo. Cambios v6.1: se agrega la **evaluación técnica del transporte POST** (§4.1.1) con la cadena de límites reales, tamaños medidos en corridas reales (1 expediente ≈ 4,2KB; tope **maxMovimientos** = 15), tabla de escenarios (individual/lote/corrida grande), la inflación por URL encoding y su mitigación, y el patrón **Post/Redirect/Get** para que la SPA reciba el payload con URL limpia. Conclusión: tras subir el límite del body, no hay límite práctico para los tamaños reales (corrida de 100 exp. ≈ 400KB entra con >10% de margen). **Revisión 2026-07-19 (Fable/Sonnet):** validada contra el código real. **3 inconsistencias corregidas** — (1) §4.1.1: Nginx **ya está en 20M** (no 1MB); la instrucción de "subir a 5m" era un downgrade — corregida a "no tocar Nginx". (2) §4.1.1 + §11.2: el fix del body limit de Express estaba mal planteado (los parsers son globales y la captura usa **urlencoded**, no **json**) — corregido con el matiz de orden de parsers. (3) §11 Fase 1.1: decía "2 columnas", el modelo §7 define **3** (**bitacora_enabled**, **home_section**, **bitacora_prefs**) — corregido. **Verificado correcto:** la claim del tour (§11 F2.7, **target: 'tab-nav'** — paso 2) coincide con **onboarding/tour.js** — real. **Agregado:** §11.1 con **modelo (Sonnet/Opus) y nivel de esfuerzo por sub-bloque**. Sigue siendo propuesta NO aprobada. **Revisión 2ª pasada 2026-07-19 (pre-ejecución): 2 correcciones adicionales que habrían hecho fallar la implementación** — (4) §7: el **UNIQUE(user_id, jurisdiccion, expediente)** con **jurisdiccion** nullable **no deduplica** (los NULL son distintos entre sí en Postgres — fichas duplicadas y el **ON CONFLICT** del upsert nunca matchearía); fix: **jurisdiccion NOT NULL DEFAULT ''** (o índice único con **COALESCE**). (5) §4.1.1: se explicita que **POST /usuarios/capture** llega **sin autenticación** (un form no puede mandar **Authorization**; el diseño "sin tokens en el payload" es correcto) — se especifican las 5 protecciones obligatorias del patrón borrador-anónimo — reclamo autenticado (id no adivinable, TTL+uso único+tope, rate-limit dedicado; payload no confiable que no persiste hasta confirmar, punto de montaje antes del static con parser propio). **Revisión 2026-07-25 (Opus; verificación de cohesión contra el código real y el plan de fases): 6 hallazgos técnicos.** (6) §2/§4.4 — la premisa "todos los visores inyectan **DATOS_BATCH** vía **generador_visor.js**" es **FALSA**: solo el visor de **informe_batch** usa ese mecanismo; los visores de **procuración** (individual, por fecha, lote) los generan los **scripts encriptados** (**procesarNovedadesCompleto.js**; **procesarCustomExpedientes.js**), que inyectan los datos con un placeholder distinto (**datosEmbebidos** / **<!-- DATOS_EMBEBIDOS -->**) — **main.js** no controla ese payload. Corregido: la botonera HTML se agrega igual (vía **visorModal_template.html**, que va como **extraResources**, sin tocar scripts), pero los datos por usuario (**bitacoraEnabled**, marcado de "casos ya seguidos") requieren que **main.js** **post-procese el HTML ya generado** antes de abrirlo (inyectando un **<script>** con la config), no una edición de los scripts encriptados. Sin riesgo de tocarlos, pero es un paso más en F2.1. (7) **Falta ABM de feriados:** Q11 dice "el admin los mantiene" pero ninguna fase lo construye — agregado F1.8. (8) **capture=lote sin tope de filas:** el rate limit (30/5min) limita frecuencia, no volumen por request; agregado cap explícito de 200 casos/lote. (9) **Recorte 2+2 no atómico:** "DELETE del más viejo... lógica de aplicación" puede dejar 3+ snapshots con capturas simultáneas del mismo caso; debe ir en la misma transacción del insert. (10) **Contradicción de gating:** §8 dice "middleware en TODOS los endpoints... es el gate real" pero §5.3/Q6 prometen exportación disponible 90 días tras perder el plan — incompatibles sin un carve-out explícito en el middleware para **/bitacora/export**. (11) Referencias desactualizadas: **server.js:84/87** son hoy **110/113**; la opción (b) del body limit (subir el **express.urlencoded** global) queda descartada, no como alternativa — reabrirla el problema que motivó el cap de C5 (revisión de bugs 2026-07-25); y **/usuarios/capture** **no debe agregarse a PUBLIC_OPEN_CORS_PATHS** (array que no existía cuando se escribió esta propuesta, agregado luego para **/analytics/event**) — el capture es una navegación de formulario, no un **fetch**, no necesita esa entrada. **6 hallazgos de cohesión del plan de fases** (mismo repaso): (12) §11 Fase 1 (7 puntos) y §11.1 (7 sub-bloques F1.1-F1.7) no mapean 1:1 — el punto 3 de §11 se parte en F1.3+F1.4, los puntos 4-5 se juntan en F1.5 — reenumerados los puntos de §11 para que coincidan exactamente con §11.1. (13) El endpoint **capture** (con su parser de 5MB, rate-limit dedicado y PRG) se construía en Fase 1 pero su único consumidor (los visores) es Fase 2 — es además el único endpoint anónimo de todo el sistema: **se movió a Fase 2**, junto a quien lo usa. (14) El deliverable de Fase 2 decía "un release de Electron" pero el punto 3 (**GET /client/bitacora/seguidos**) es un endpoint de backend nuevo — aclarado: Fase 2 = deploy de backend + release de Electron.

(15) La salvaguarda de importación (backup automático antes de aplicar, \$5.3) depende de que la exportación ya funcione — F1.6 pasa a ser dependencia dura de F1.7, no solo orden sugerido. (16) \$9.3 y \$11.1 no cerraban entre sí (\$11.1 le da a un solo sub bloque, F1.3, el presupuesto que \$9.3 había dado a Fase 1 portal completa) — reconciliado, total realista **10-14 sesiones**, no 7-10. (17) Nota de producto agregada: la Fase 1 sola no tiene diferencial (la propuesta de valor —"un clic desde la procuración"— es Fase 2) → no anunciar/vender la Bitácora al cerrar Fase 1, esa fase es para validación interna con el flag encendido en un plan de prueba. Detalle en [docs/internal/revisión-bugs-2026-07-25c.md](#) . Sigue siendo propuesta NO aprobada.

0. Autosuficiencia del documento y fuentes

Este documento es autosuficiente para evaluar y para implementar la mejora: todo lo que se tomó como inspiración externa ya está volcado acá adentro (los tipos de agendamiento, el esquema de campos, el mecanismo de avisos y el concepto de gestiones provienen del manual de Lex-Doctor 11 y están adaptados y especificados en §3; los patrones de interacción de calendario y lista de tareas provienen de Google Calendar/Tasks y están descriptos en los mockups y flujos). **No hace falta tener a mano ninguna fuente externa para trabajar con este plan.**

Referencias, por si se quisiera profundizar (opcional, no requerido):

- **Manual de Lex-Doctor 11** (convertido a Markdown): [C:\Users\JONATHAN\Desktop\variete\man_usuario_lex11.md](#) — útil solo si a futuro se quisiera inspirar OTRO módulo (caja, facturación a clientes, modelos de escritos); para la Bitácora ya no aporta nada que no esté acá. Secciones usadas: Agenda (§XIII), Gestiones (§XIV).
- **Google Calendar / Google Tasks:** conceptos de UX públicos y consultables en internet en cualquier momento; no se usa ninguna API ni servicio de Google (descartado en §12), solo el patrón de interacción.

1. Resumen ejecutivo

Se propone incorporar a Procurador SCW un módulo de organización del trabajo del abogado —nombre: **Bitácora**— compuesto por dos piezas que se alimentan mutuamente:

1. **Entradas de bitácora:** vencimientos, audiencias, tareas, gestiones y notas, con vista calendario + lista de pendientes estilo Google Calendar/Tasks, gestionadas desde el portal del usuario.
2. **Expedientes seguidos:** una ficha liviana por caso (expediente, jurisdicción, dependencia, carátula, situación actual) que se crea automáticamente la primera vez que el usuario genera una entrada de bitácora sobre ese caso desde un visor, y que acumula las entradas siguientes. Cada ficha guarda un **historial acotado: hasta 2 procuraciones y hasta 2 informes** del caso (los más recientes, con qué se obtuvo en cada uno), para no saturar la base. La captura desde el visor funciona en dos niveles: **selección múltiple** en la tabla general (varios casos de una vez) y **captura individual** desde la fila o el modal de cada caso; los casos ya guardados muestran un **link a su ficha** con todo lo registrado.

El circuito completo: el abogado procura → el visor HTML muestra los resultados → con un clic en un expediente crea un vencimiento/tarea/nota → esa acción crea (o actualiza) la ficha del caso en su panel, con la foto de la procuración → desde el portal consulta el calendario, confirma lo realizado con un check, edita la ficha o elimina el seguimiento del caso cuando terminó.

Diferenciador: en Lex-Doctor y similares la agenda y los expedientes se cargan 100% a mano. Acá el dato nace de la automatización que ya corre todos los días — el expediente se "sigue solo" a partir del primer uso real.

Puntos fijados en esta versión:

- Captura desde visores por **deep-link al portal** (opción A de la v1) — sin tokens embebidos, sin CORS nuevo.
- Visores que capturan: **procuración individual, procuración batch, informe individual e informe batch** (el informe individual hoy no genera visor — ver §5.3, se propone generar un mini-visor reutilizando el template existente).
- **Avisos solo dentro de la Bitácora** (banner superior al ingresar), sin emails.
- **Check de realización** en vencimientos y tareas; los vencidos sin confirmar se muestran hasta 7 días hacia atrás (con "ver anteriores") y los próximos 7 días hacia adelante (con "ver más").
- **Bitácora habilitable por plan** (gating comercial), lo que activa/oculta las opciones tanto en los visores como en el portal.
- **Sin integración con Google** por ahora.

2. Contexto actual (qué ya tenemos y se reutiliza)

Pieza existente	Rol en la propuesta
Visores HTML (<code>visorModal_template.html</code> , visor de informes)	Punto de captura. Ya inyectan un JSON con expediente, carátula, dependencia, situación y movimientos — todo lo necesario para pre-cargar la ficha del caso y la entrada. ⚠ Dos mecanismos distintos (ver §4.4): el visor de informe batch lo genera <code>generador_visor.js</code> (Electron) con la variable <code>DATOS_BATCH</code> ; los visores de procuración los generan los scripts encriptados con la variable <code>datosEmbebidos</code> — <code>main.js</code> no controla ese payload directamente.
Portal <code>/usuarios/</code> (SPA vanilla, secciones por <code>goto=</code> , SSO desde Electron)	Casa del módulo: secciones Bitácora y Mis expedientes .
Backend Express + PostgreSQL	3 tablas nuevas + endpoints CRUD. Sin dependencias nuevas.
Notificaciones in-app (<code>user_notifications</code>)	No se usan para los avisos de bitácora (los avisos viven en el banner de la propia Bitácora), pero quedan disponibles para hitos puntuales si hiciera falta.
Tabla <code>plans</code> + <code>/client/account</code>	Gating por plan: flag <code>bitacora_enabled</code> expuesto a la app y al portal.
App Electron (topbar, <code>openPortalSection</code>)	Acceso rápido a la Bitácora del portal vía SSO + generación condicional de los botones en los visores.

3. Modelo funcional

3.1 Expediente seguido (la ficha del caso)

Es la unidad central del módulo. Contiene:

Campo	Origen	Editado por el usuario
Expediente (número/año)	del visor (o carga manual)	Sí
Jurisdicción	del visor	Sí
Dependencia	del visor	Sí
Carátula	del visor	Sí
Situación actual (última registrada)	del visor — se actualiza con cada nueva captura sobre el caso	Sí
Fecha de la situación	fecha de la corrida que la registró	—
Notas del caso	libre	Sí

Reglas de vida de la ficha:

- **Creación por dos vías:** (a) **automática**, la primera vez que el usuario genera una entrada de bitácora (o guarda el caso) desde un visor; (b) **manual**, desde el botón " + Agregar " de Mis expedientes — formulario con los mismos campos de la ficha (expediente, jurisdicción, dependencia, carátula, situación, notas). Un caso creado a mano es idéntico a uno capturado: si después se lo captura desde un visor, el upsert lo reconoce por la clave jurisdicción+expediente y le suma el snapshot (no duplica). Sirve para casos que el abogado quiere agendar antes de haberlos procurado nunca.
- **Acumulación:** si ya existe la ficha (misma jurisdicción + expediente), una nueva captura **no duplica**: agrega la entrada de bitácora a la ficha existente, actualiza la situación actual y suma el snapshot al historial.
- **Historial acotado — hasta 2 de cada tipo, POR CASO (definición precisa):** el tope es **por expediente seguido**, no global ni por corrida. Cada caso guardado conserva **su propio** historial: como máximo sus últimas 2 procuraciones y sus últimos 2 informes. Lo que se guarda no es "la corrida" entera sino **la porción de esa corrida que corresponde a ese caso** (su situación, sus movimientos): una corrida de lote con 30 expedientes alimenta con un snapshot a cada caso seguido que aparezca en ella, cada uno por separado. Ejemplo: el caso A tiene guardadas sus procuraciones del 05/07 y del 28/06 y un informe del 15/06; el caso B, procurado en las mismas corridas, tiene las suyas propias; el caso C, capturado solo desde informes, tiene 2 informes y ninguna procuración. Es un tope, no una garantía: cada caso tiene 0, 1 o 2 de cada tipo según cómo se lo haya usado. Al entrar un tercer snapshot del mismo tipo **en ese caso**, se elimina el más viejo **de ese caso**. La base queda liviana de forma estructural (máximo 4 snapshots por caso), sin crons de limpieza.
- **Edición:** todos los campos editables desde la ficha en el portal.

- **Eliminación del seguimiento:** botón "Eliminar seguimiento" con confirmación. Al eliminar, el usuario elige si las entradas de bitácora vinculadas **se conservan como sueltas** (default, no pierde sus vencimientos) o **se eliminan también**.

3.2 Entradas de bitácora

Tipo	Color	Fecha	Check de realización
Vencimiento	rojo	obligatoria	Sí
Audiencia	violeta	obligatoria	Sí
Tarea	azul	opcional	Sí
Gestión	ámbar	opcional (salida/regreso)	Sí
Nota	gris	fecha de creación	No (es un registro, no un pendiente)

- Toda entrada puede estar **vinculada a un expediente seguido** o ser **suelta** (recordatorio general del estudio). Las vinculadas viven en **dos vistas a la vez**: aparecen en la **ficha del caso** (junto con sus datos y su historial — la vista "por caso") y en el **calendario/lista de la Bitácora** (mezcladas con las de todos los casos y las sueltas — la vista "por tiempo"). Son los mismos registros vistos con dos lentes: la ficha responde "¿qué pasa con este expediente?", el calendario responde "¿qué tengo que hacer esta semana?".
- **Check de realización:** un clic marca la entrada como hecha (guarda fecha/hora de confirmación). Se puede deshacer. Es el mecanismo central de control: lo no chequeado sigue reclamando atención en el banner de avisos.
- **Repetición** simple (semanal/mensual/anual) para recordatorios fijos.
- **Calculadora de plazos procesales:** al crear un vencimiento, opción "hoy + N días hábiles" que calcula la fecha saltando fines de semana y feriados (tabla de feriados nacionales AR + ferias judiciales, editable por el admin). Feature de altísimo valor percibido para el destinatario (abogado/operador judicial) y barata de construir.

Campos por tipo de entrada (definición)

Referencias adaptadas: Lex-Doctor 11 define el agendamiento con tipo (Tarea/Audiencia/Compromiso/Vencimiento), estado hecho/no hecho, fecha y hora, repetición, descripción, responsable, proceso vinculado y aviso con anticipación; Google Calendar/Tasks aportan el patrón título + fecha/hora + todo-el-día + repetición + notas. Lo siguiente es la síntesis orientada a la procuración jurídica, para una cuenta de un solo abogado (sin campo "responsable" en v1).

Campos comunes a todos los tipos:

Campo	Detalle
Título	obligatorio, corto (ej. "Contestar traslado")
Descripción	libre, opcional; al capturar desde un visor viene pre-cargada con el movimiento
Expediente vinculado	opcional — autocompletar sobre Mis expedientes, o vacío (entrada suelta)
Estado	pendiente / hecho, con fecha-hora de confirmación (el check) — no aplica a Nota
Origen	manual / visor de procuración / visor de informe (automático, no editable)

Campos específicos por tipo:

Tipo	Campos propios	Notas de uso jurídico
Vencimiento	Fecha límite (obligatoria, con calculadora de días hábiles) · Aviso: días de anticipación (default 5) · Carácter: procesal / extraprocesal (opcional)	El tipo central de la práctica: cargas procesales, plazos de recursos, caducidades. La fecha calculada muestra el detalle ("hoy + 5 hábiles = 14/07, saltea feria")
Audiencia	Fecha y hora (no es todo-el-día) · Lugar / sala (texto) · Modalidad: presencial / virtual (con campo para el link) · Aviso (default 3 días)	Testimoniales, conciliaciones, vistas de causa
Tarea	Fecha opcional (sin fecha = pendiente permanente en el panel de tareas)	Preparar escritos, armar prueba, llamar al cliente — el gesto Google Tasks
Gestión	Fecha prevista · Organismo / lugar (texto: "Juzgado Federal 2", "Colegio de Abogados") · Estado ampliado: pendiente / realizada / a reintentar	Trámite presencial o extraprocesal (retirar cédula, diligenciar oficio, presentación en mesa de entradas) — tomado de Gestiones de Lex-Doctor, simplificado
Nota	Solo título + texto (fecha = la de creación; sin check)	Registro de bitácora puro: "hablé con el cliente, acepta el acuerdo"

- Los campos específicos se guardan en una columna flexible (`meta JSONB`) — agregar un campo futuro a un tipo no requiere migración.
- **Edición global y por caso — mismas entradas, dos puertas:** una entrada se puede editar tanto desde la vista global de la Bitácora (calendario/lista) como desde la ficha de su expediente; es **el mismo registro** — el cambio se ve al instante en ambas vistas. Además, la vista de lista permite **acciones masivas:** tildar varias entradas y marcarlas hechas / eliminarlas / moverlas de fecha en un solo paso.

3.3 Avisos y recordatorios (solo in-app, sin email)

Los avisos viven **dentro de la Bitácora**, en un banner superior que se muestra al ingresar a la sección (y por lo tanto al iniciar sesión, si la Bitácora es la pantalla principal):

⚠ VENCIDOS SIN CONFIRMAR (3)

[ver anteriores]

☐ 01/07 Vencimiento - Contestar traslado · FCR 1234/2021

☐ 03/07 Tarea - Retirar cédula · CIV 887/2023

☐ 04/07 Vencimiento - Apelar honorarios · (suelta)

📅 PRÓXIMOS 7 DÍAS (2)

[ver más]

☐ 08/07 Audiencia - Testimonial · FCR 1234/2021

☐ 11/07 Vencimiento - Ofrecer prueba · COM 456/2024

- **Vencidos sin confirmar:** entradas con fecha pasada y sin check, mostrando por defecto **hasta 7 días de antigüedad**; el botón "ver anteriores" expande la ventana (14/30 días/todos). Nada desaparece solo: lo que no se checkeó sigue existiendo, solo que colapsado para no abrumar.
- **Próximos:** ventana default de **7 días hacia adelante**, con "ver más" (14/30 días).
- El **checkbox está en el propio aviso:** confirmar una realización es un clic, sin abrir la entrada.
- Sin emails ni notificaciones push: el abogado ve sus vencimientos al entrar, que es el hábito que el producto quiere construir. (Si más adelante se pide un resumen por email, la infraestructura de `mailer.js` existe — queda explícitamente fuera de esta versión.)

3.4 Pantalla principal configurable (píldora "Establecer como principal")

En el portal, las secciones **Mi Plan** y **Bitácora** llevan arriba una píldora/toggle:

Mi Plan

[★ Es tu pantalla principal]

← estado activo

Bitácora

[☆ Establecer como principal]

← estado disponible

- Son **mutuamente excluyentes:** activar una desactiva la otra (una sola preferencia `home_section` por usuario).
- Efecto: al iniciar sesión en el portal (o al entrar por SSO sin `goto=` explícito), se abre la sección elegida.
- Default para usuarios existentes: Mi Plan (comportamiento actual, sin sorpresas).

4. Captura desde los visores — diseño técnico

4.1 Mecanismo: deep-link al portal, por formulario POST (opción A revisada)

Los visores son HTML estáticos abiertos con `file://`, sin sesión. **Se descartó el link GET con datos en el querystring** (ver el porqué abajo) a favor de un **formulario HTML oculto que se autoenvía por POST** al hacer clic:

```
<form id="pscBitacoraForm" method="POST"
  action="https://api.procuradortool.com/usuarios/capture"
  target="procurador_portal" style="display:none">
  <input name="goto" value="bitacora-nueva">
  <input name="tipo" value="vencimiento">
  <input name="exp" value="FCR 1234/2021">
  <input name="jur"> <input name="dep"> <input name="car">
  <input name="sit"> <input name="fproc" value="2026-07-05">
  <input name="origen" value="procuracion">
  <input name="movs"> <!-- JSON de movimientos, sin recortar -->
</form>
<script>document.getElementById('pscBitacoraForm').submit();</script>
```

y el navegador navega la pestaña `procurador_portal` a ese POST. El portal (con sesión activa, o tras login) abre el modal "Nueva entrada" **pre-cargado** con los datos del caso y el snapshot completo de la corrida.

- **Por qué POST-formulario y no GET-link (el cambio de esta versión):** un envío de `<form>` es una **navegación de página completa** —igual que hacer clic en un link—, **no** una llamada `fetch` /AJAX; por eso **no dispara CORS** aunque el origen sea `file://null`, exactamente la misma propiedad que tenía el link GET. La diferencia es dónde viajan los datos: un GET los lleva en la **URL** (limitada a ~2.000 caracteres prácticos); un POST los lleva en el **body de la petición**, sin ese límite (el único tope pasa a ser el `body-parser` del servidor, configurable a varios MB — trivial para cualquier snapshot real). **Resultado: desaparece la necesidad de recortar movimientos o de limitar la selección múltiple a 10 casos** (ver §4.2, corregido más abajo). El único costo es que el botón dispara un formulario invisible en vez de ser un link — imperceptible para el usuario, sigue siendo "un clic".
- **Sin token embebido en el HTML** (que sería compatible por error) — el POST no necesita transportar credenciales, solo los datos del caso ya conocidos por el usuario.
- Cuando el visor se abre automáticamente desde la app (flujo principal), el formulario puede llevar además el hash SSO como ya hace `openPortalSection` → el usuario cae **logueado, con el modal abierto y los campos completos**: 1 clic + guardar.
- Si el usuario reabre el HTML días después desde la carpeta, pasa por el login del portal y sigue el mismo flujo (aceptable).
- **Enlaces livianos siguen siendo GET simples:** no todo necesita el formulario — un link "Ver ficha" (`goto=expediente&id=...`, sin payload de datos) sigue siendo un `<a>` normal; el POST-formulario se usa solo donde viaja un snapshot o una selección múltiple.

4.1.1 Límites reales, tamaños medidos y patrón de recepción (evaluación técnica)

Como el POST reemplaza el tope de ~2.000 caracteres de la URL por el tope del body del servidor —que es **configuración nuestra**—, conviene dejar los números concretos para quien implemente.

La cadena de límites (cada capa con su tope por defecto):

Capa	Límite actual (verificado 2026-07-19)	¿Quién lo controla?
Navegador (form POST)	Sin límite práctico (a diferencia de una URL)	—
Nginx (<code>client_max_body_size</code>)	20 MB (ya configurado en prod Y staging — <code>sites-available/procurador</code> y <code>staging-procurador</code>)	Nosotros (config del server)
Express (<code>express.urlencoded</code> GLOBAL)	100 KB (default; sin <code>limit</code> : explícito — verificar el número de línea actual en <code>server.js</code> al implementar, ver nota 2026-07-25 abajo)	Nosotros

⚠ **Corrección técnica (2026-07-19).** La versión previa de esta tabla decía "Nginx = 1 MB → subir a 5m". **Es incorrecto:** Nginx ya está en **20 MB** en ambos entornos. Poner `client_max_body_size 5m`; sería un **downgrade** (de 20M a 5M) — **no tocar Nginx**, ya tiene margen de sobra para el peor caso (~400 KB).

El único tope real a subir es el de Express, y hay un matiz de orden que la versión previa omitía: los parsers `express.json` (con hook `verify` para el rawBody del webhook de MP) y `express.urlencoded` están montados **globalmente**, **antes** de todos los routers (⚠ **verificado 2026-07-25: hoy son `server.js:110` y `113`** — las líneas se corrieron respecto de cuando se escribió esta sección; confirmar el número exacto al implementar, no asumir 84/87). La captura llega como **form POST (`x-www-form-urlencoded`)**, así que el que la parsea es el `express.urlencoded` **global**, no un `express.json` de ruta. Por eso un `express.json({ limit: '5mb' })` colgado solo de la

ruta de captura **no funcionaría** (parser equivocado + corre después del global, que ya habría rechazado con 413 a los 100 KB). La forma correcta:

- **Montar un parser específico ANTES del router en `server.js`** — `app.use('/usuarios/capture', express.urlencoded({ extended: false, limit: '5mb' })))` colocado **arriba** del `express.urlencoded` global (así gana la ruta de captura y el resto del sistema sigue en 100 KB). Preserva intacto el `express.json` global con su `verify` del webhook.
- ⚠️ **Descartada (no usar): subir el límite del `express.urlencoded` GLOBAL a `5mb`**. No es una alternativa válida — reabrirla el mismo problema que motivó el cap de longitud agregado a `POST /tickets` el 2026-07-25 (hallazgo C5, `docs/internal/revision-bugs-2026-07-25.md`): un límite global generoso habilita abuso en cualquier endpoint que hoy depende de esos 100 KB por defecto como techo implícito. El parser de la ruta de captura debe ser **específico y acotado a esa sola ruta**.
- ⚠️ **No agregar `/usuarios/capture` a `PUBLIC_OPEN_CORS_PATHS`**. Ese array (en `server.js`, agregado el 2026-07-25 para `POST /analytics/event`, ver hallazgo D4 de `revision-bugs-2026-07-25b.md`) abre CORS sin credenciales para beacons cross-origin llamados por `fetch`. El capture **no lo necesita**: es una navegación de `<form>` (POST tradicional, no AJAX), que nunca dispara preflight/CORS sin importar el origen — agregarlo ahí no rompe nada pero es una entrada muerta que puede confundir a quien lea la lista después.

Tamaños reales medidos (corridas del CUIT de prueba 27320694359):

Unidad	Tamaño real
1 movimiento	~170–380 bytes (mediana ~230)
1 expediente completo (15 movimientos — tope <code>maxMovimientos</code> de la config)	~4,2 KB
Corrida de 3 expedientes (JSON entero)	~19 KB

Escenarios contra los límites:

Escenario	Payload estimado	Veredicto
Captura individual (1 caso)	~4 KB (peor caso realista con 100 movimientos: ~25 KB)	✅ Entra incluso con el default de 100 KB
Lote ~10 casos con snapshot	~40 KB	✅ Entra en 100 KB
Lote ~20–25 casos con snapshot	~80–100 KB	⚠️ Roza el default → subir el límite
Corrida grande (50–100 exp. con snapshot)	~200–400 KB	❌ Excede 100 KB → requiere subir el límite (a 5 MB entra con >10× de margen)

La inflación por URL-encoding (a tener en cuenta): un `<form>` por defecto codifica en `application/x-www-form-urlencoded`, y como el JSON está lleno de `{ } " : ,` y acentos, el encoding **infla ~2–3×** (cada carácter especial → `%XX`). Un JSON de 40 KB puede viajar como ~100 KB. **Mitigación:** **subir el límite del `express.urlencoded` de la ruta de captura a 5 MB** (ver el recuadro de corrección arriba — se monta un parser específico antes del router, no se toca Nginx que ya está en 20M). Holgadísimo, sin costo. Alternativa innecesaria: `enctype="text/plain"` para no inflar.

Patrón de recepción — Post/Redirect/Get (PRG): el POST aterriza en el portal (una SPA); para que la SPA reciba los datos de forma limpia:

1. El visor hace POST a `/usuarios/capture` con el payload.
2. El servidor guarda el payload un instante (sesión o registro efímero con id) y responde con **303 redirect** a la URL limpia del portal (`/usuarios/?goto=bitacora-nueva&draft=<id>`).
3. La SPA carga, lee `draft=<id>`, pide el payload al backend y abre el modal pre-cargado.

Ventajas del PRG: evita el "¿reenviar formulario?" al refrescar, deja la URL limpia (el payload no queda en el historial del navegador), y el límite es 100% server-side. Es exactamente cómo funcionan los SSO/SAML por POST binding (mandan tokens de varios KB así) — técnica estándar y de bajo riesgo.

⚠️ **Precisión de seguridad agregada 2026-07-19 — el POST de captura llega SIN autenticación (diseñarlo así a propósito, con estas protecciones):** un `<form>` HTML no puede mandar el header `Authorization`, y la decisión de diseño (riesgo #3 de §12) es correcta: **sin tokens en el payload** (el visor es un archivo local que podría compartirse). Consecuencia que el implementador debe tener explícita: `POST /usuarios/capture` **no puede llevar `authenticateToken`** — es un endpoint público que recibe un borrador **anónimo**, y la autenticación ocurre recién en el paso 3 del PRG, cuando la SPA logueada **reclama** el draft (`GET /usuarios/api/capture-draft/:id`, ese Sí con JWT). Para que eso no sea un agujero, el endpoint público necesita, como mínimo:

1. **Id de draft no adivinable** (`crypto.randomBytes(32)`) — mismo patrón que los tokens de verificación de email ya usados en `auth.js`, devuelto SOLO en el redirect.
2. **TTL corto** (ej. 10 minutos) + borrado al reclamar (uso único) + límite de drafts simultáneos en memoria/tabla (ej. 100) para que no sea un sumidero de memoria.

3. **Rate-limit** dedicado (patrón `rateLimiter.js`, ej. 30/5min por IP) — es el único endpoint anónimo nuevo que agrega la Bitácora.
4. **Payload tratado como entrada NO confiable**: se valida estructura/tamaño al reclamar y **nada se persiste en tablas reales hasta que el usuario autenticado confirma** en el modal (el draft es un buffer, no un insert).
5. **Punto de montaje**: la ruta se monta en `server.js` (ej. `app.post('/usuarios/capture', ...)`) **antes** del `express.static` de `/usuarios` (los estáticos solo atienden GET, pero el orden explícito evita sorpresas), y con su parser `urlencoded({ limit: '5mb' })` propio (ver corrección de límites arriba).

Veredicto: complejidad baja (form oculto + `.submit()` en el visor; una línea de límite en Express y Nginx; PRG en el endpoint). Tras el ajuste de límite, **no hay límite práctico** para los tamaños de datos reales — una corrida de 100 expedientes (~400 KB) entra con margen enorme en 5 MB. El único límite que queda es de **usabilidad** (revisar decenas de filas antes de guardar), no técnico → por eso la pantalla de revisión del lote pagina/agrupa (§4.2), pero el transporte ya no impone tope.

Una sola pestaña del portal + botón Volver coherente. El caso típico es trabajar el visor y disparar varias capturas seguidas — no puede ser que cada clic abra una pestaña nueva del portal. Diseño:

- **Pestaña con nombre fijo**: todos los links de bitácora del visor llevan `target="procurador_portal"` (nombre de ventana fijo). El navegador **reutiliza la misma pestaña** en cada clic: el primer deep-link la abre, los siguientes navegan en ella y le dan foco. Solo si el usuario la cerró se abre una nueva.
- **Sesión en uso, no re-login**: si la pestaña del portal ya tiene sesión activa, el deep-link **no vuelve a loguear** — el hash SSO se consume solo cuando no hay sesión; con sesión viva, el portal simplemente navega al destino (`goto=`) con los datos pre-cargados.
- **Volver = pantalla anterior de la Bitácora**: el portal ya navega entre secciones con la History API (patrón implementado en ambos dashboards para el botón Atrás). Cada pantalla del flujo de bitácora (modal de nueva entrada, ficha, lote) **apila su estado en el historial del navegador** → después de guardar la segunda entrada, "Volver" regresa a la pantalla de bitácora anterior (ej. la confirmación o ficha del caso previo), no expulsa al login ni pierde el contexto.
- **Deep-links repetidos, resultado idempotente**: si el usuario dispara dos veces el mismo link (doble clic, impaciencia), el upsert por clave del caso y el formulario pre-cargado (que no guarda hasta confirmar) evitan duplicados.

4.2 Visores alcanzados y qué se agrega en cada uno

Los cuatro visores que capturan son: **procuración individual, procuración batch, informe individual, informe batch**. (El visor del monitor queda fuera de esta versión — evaluable después.)

La captura tiene **dos niveles**, pensados para los dos momentos de trabajo sobre el visor:

a) Tabla general — selección múltiple (trabajo en lote)

La tabla del visor suma una **columna de checkboxes** y una barra de acciones que aparece al seleccionar:

☒ 3 seleccionados [🔥 Guardar casos] [📄 Guardar procuración] [+ Crear entradas...] [X limpiar]					
☒	FCR 1234/21	PEREZ c/ GOMEZ s/...	Juzg. Fed. 2	DESPACHO	📁 ficha
☒	CIV 887/23	RUIZ c/ BANCO s/...	Juzg. Civ. 4	SENTEN.	[📄 +]
☐	COM 456/24	ACME SA s/ CONCURSO	Juzg. Com. 1	TRASLADO	[📄 +]
☒	FCR 555/22	LOPEZ c/ ESTADO s/...	Juzg. Fed. 2	DESPACHO	[📄 +]

Tres acciones independientes sobre la selección (**ficha básica y snapshot** ya no van pegados — corrección de esta versión):

- "🔥 **Guardar casos**": crea/actualiza **solo la ficha** (expediente, jurisdicción, dependencia, carátula, situación actual) de todos los seleccionados — **sin** tocar el historial de snapshots. Para el que quiere empezar a seguir varios casos sin comprometerse a nada más.
- "📄 **Guardar procuración de seleccionados**" (en visores de informe: "📄 **Guardar informe de seleccionados**"): crea/actualiza la ficha **y además** guarda el snapshot de esta corrida (con sus movimientos) en el historial de cada seleccionado. Es "Guardar casos" + adjuntar el snapshot en un solo paso, para el que ya sabe que quiere ambas cosas.
- " + **Crear entradas...**": elige un tipo (Vencimiento/Tarea/Nota) y abre en el portal una **pantalla de revisión del lote**: una fila editable por caso (título pre-cargado, fecha con la calculadora de plazos aplicable a todos o por fila) → "Guardar todo". Sirve al caso real de "estos 5 tienen traslado, les pongo vencimiento a todos".
- **Sin límite artificial de selección**: al viajar por POST (§4.1) y no por querystring, ya no hace falta topear la selección a 10 casos ni recortar los movimientos del lote — el usuario puede tildar todos los expedientes de un batch grande. El único límite razonable es de **usabilidad**, no de transporte: la pantalla de revisión de "Crear entradas..." pagina o agrupa si el lote es muy largo (ej. > 30 filas), para que siga siendo cómoda de revisar antes de guardar.
- ⚠️ **Cap técnico obligatorio en el backend (revisión 2026-07-25, hallazgo H3)**: "sin límite" es correcto para la experiencia de usuario, pero `POST /usuarios/api/expedientes/capture-lote` necesita un **tope explícito de filas por request** independiente del límite de

bytes (5 MB). El rate-limit propuesto en §4.1.1 (30/5min) limita *frecuencia*, no *volumen*: un solo POST bien armado dentro del límite de bytes podría intentar crear miles de fichas en una sola transacción. Tope sugerido: **200 casos por request**, con 400 claro si se excede ("Máximo 200 casos por lote; dividí la selección"). 200 filas de snapshot completo (~4,2 KB c/u) son ~840 KB — cómodo dentro de los 5 MB, y muy por encima de cualquier lote real de trabajo.

b) Fila / modal del caso — captura individual (trabajo caso por caso)

En cada fila, el botón compacto  despliega el mini-menú:



- Las tres primeras opciones abren el formulario POST con `tipo=` correspondiente → modal de entrada pre-cargado (viaja el snapshot completo con movimientos, §4.1, sin recorte). Al guardar la entrada, el snapshot también se suma al historial del caso — capturar una entrada ya implica guardar la procuración/informe de ese momento.
- "  **Guardar caso** " crea/actualiza **solo la ficha** del expediente (datos básicos, sin snapshot) — para el que quiere empezar a seguirlo sin comprometer todavía el historial. Toast de confirmación, sin más fricción.
- "  **Guardar procuración** " (o "  Guardar informe", según el visor) hace lo mismo que "Guardar caso" **y además** adjunta el snapshot de esta corrida al historial — para el que ya sabe que quiere conservar el resultado, aunque hoy no tenga nada que agendar.
- **En el modal de movimientos** del visor se repite la misma botonera, incluidos ambos botones de guardado para ese caso puntual; en ese contexto, el movimiento que se está mirando viaja pre-cargado como descripción de la entrada ("Nuevo despacho del 04/07: 'Traslado a la actora...' → vencimiento").

c) Casos ya seguidos — link a la ficha

Al generar el visor, la app consulta un endpoint liviano (`GET /client/bitacora/seguidos` → lista de claves expediente+jurisdicción del usuario) y marca en `DATOS_BATCH` los casos que **ya están guardados**. En el visor:

- La fila muestra  **ficha** en lugar de (o junto a)  — un link directo a la ficha del caso en el portal (`goto=expediente&id=...`), donde está todo lo ya registrado: entradas, historial de snapshots, notas.
- En el modal del caso, un encabezado sutil: "  Este caso está en tu Bitácora — Ver ficha".
- El  sigue disponible igualmente (agregar una nueva entrada a un caso seguido es el flujo de acumulación normal).
- **Nota de vigencia:** el marcado refleja el estado al momento de generar el visor; si el usuario guarda el caso después, un visor viejo no lo sabe — aceptable (el upsert del portal resuelve igual sin duplicar).

Un pie discreto en el visor: "  Bitácora — tus vencimientos y casos en el portal" con link directo a la sección, que sirve de descubrimiento de la feature.

4.3 El caso especial: informe individual (hoy sin visor)

El informe individual genera un PDF directamente (script encriptado `informequickscwpjn.js`) y no produce visor. Dos alternativas evaluadas:

Alternativa	Evaluación
A. Mini-visor para el informe individual (recomendada)	Reutilizar <code>generador_visor.js</code> + template con un array de 1 expediente. Se genera desde <code>main.js</code> (Electron) con los datos que el flujo de informe ya tiene — sin tocar el script encriptado . El mini-visor muestra la ficha del expediente + botonera de bitácora + link "Abrir PDF". Respeta la config <code>visor.abrirAutomaticamente</code> existente. Consistencia total: los 4 flujos terminan igual (visor → captura).
B. Link dentro del PDF	Insertar un hyperlink en el PDF requiere tocar el script encriptado que lo genera (re-encriptar y redeployar), el link en PDF es menos visible/accionable, y no puede desplegar el mini-menú de tipos. Además el PDF suele imprimirse/enviarse al cliente — un link interno del estudio ahí es ruido.

Recomendación: A. Costo bajo (el generador y el template ya existen), cero riesgo sobre los scripts encriptados, y unifica la experiencia. El PDF queda como el entregable "para afuera" y el visor como la superficie de trabajo "para adentro".

4.4 Gating por plan en los visores

⚠ **Corrección de arquitectura (revisión 2026-07-25, hallazgo H1) — leer antes de implementar F2.1.** Esta sección (y el resto del documento hasta esta revisión) asumía que **todos** los visores comparten el mismo mecanismo de inyección de datos (`DATOS_BATCH` , controlado por `main.js`). **Es falso.** Verificado contra el código real: hay **dos arquitecturas de visor distintas**:

Visor	Quién lo genera	Variable / placeholder
Informe batch	<code>generadorvisor.js</code> (Electron), invocado desde <code>main.js</code>	<code>DATOS_BATCH</code> — <code>main.js</code> controla el payload directamente
Procuración (individual, por fecha, lote)	Scripts encriptados (<code>procesarNovedadesCompleto.js</code> , <code>procesarCustomExpedientes.js</code>)	<code>datosEmbebidos</code> , inyectado con <code>template.replace('<!-- DATOS_EMBEBIDOS -->', ...)</code> dentro del propio script — <code>main.js</code> no controla este payload

Consecuencia: la botonera HTML/JS de la Bitácora (los botones `+` , el mini-menú, la barra de selección múltiple) se agrega igual en los cuatro visores, editando `visorModal_template.html` y el template del informe — ambos son archivos planos del repo Electron (`visorModal_template.html` va como `extraResources` del build, confirmado en `package.json`), **sin tocar ningún script encriptado**. Eso sigue siendo cierto y de riesgo bajo.

Lo que **no** puede hacerse como estaba planteado es que `main.js` inyecte `bitacoraEnabled` (o cualquier dato dependiente del usuario, como el marcado de "casos ya seguidos" de §4.2c) **dentro** del payload de los visores de procuración, porque ese payload lo arma el script encriptado, no `main.js` .

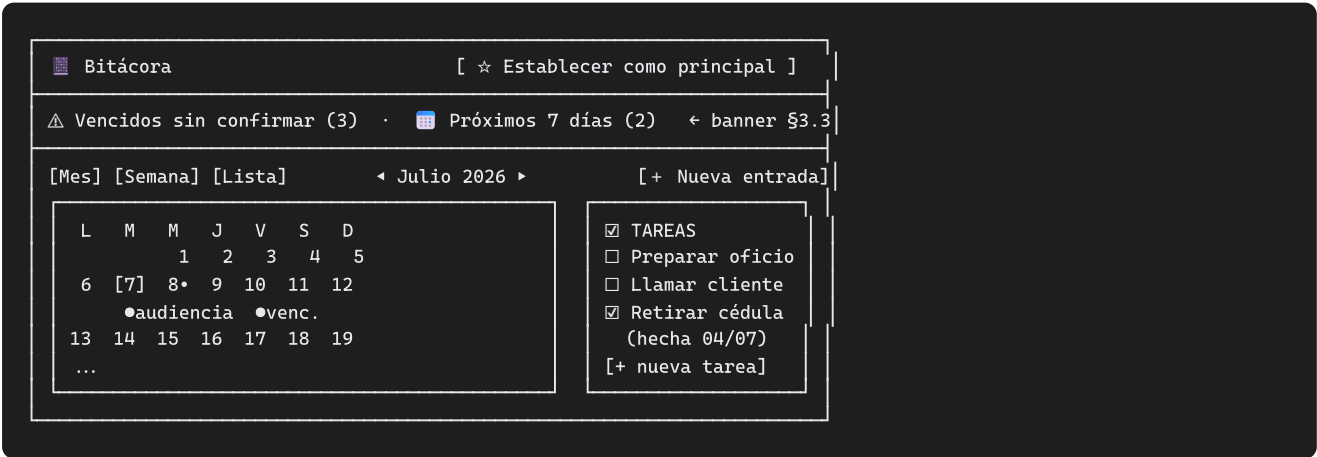
Solución adoptada — post-procesado del HTML ya generado: para los visores de procuración, en vez de inyectar datos *durante* la generación (que ocurre dentro del script encriptado), `main.js` **post-procesa el archivo HTML después de que el script lo generó y antes de abrirlo** — inserta un `<script>` adicional al final del `<body>` con `bitacoraEnabled` y la lista de casos ya seguidos (obtenida de `GET /client/bitacora/seguidos` , §4.2c), usando el mismo patrón de `fs.readFileSync` + `string.replace` + `fs.writeFileSync` que `main.js` ya usa hoy para otras tareas de post-proceso de archivos generados. El script del visor (JS estático de `visorModal_template.html`) lee esa variable igual que leería `DATOS_BATCH` . **Cero cambios en los scripts encriptados, cero re-encriptado, cero redeploy de scripts** — el costo es un paso adicional (post-procesado) en el flujo de `main.js` para los 3 visores de procuración, que sí es trabajo nuevo respecto de lo que decía la versión anterior de este documento (afecta la estimación de F2.1, ver §11.1).

Con esa corrección, el resto de esta sección se mantiene:

- **Habilitado** → botonera visible.
- **Deshabilitado** → sin botones; opcionalmente un pie sutil " `Bitácora disponible en planes superiores`" (palanca de upsell, a decidir comercialmente).
- El gate real está **en el backend** (los endpoints de bitácora rechazan con 403 si el plan no la incluye); lo del visor es solo presentación. Un visor viejo generado cuando el plan la incluía muestra botones, pero el portal responde correctamente según el plan vigente.

5. El panel del usuario — qué ve y cómo se navega

5.1 Sección "Bitácora" (nueva)



- **Barra de filtros** (arriba de las vistas, persistente entre visitas):

- **Chips por tipo** con su color: [Todos] [● Venc.] [● Aud.] [● Tareas] [● Gest.] [● Notas] — prender/apagar tipos con un clic (en el calendario, apaga los puntos de ese color; en la lista, oculta la sección).
- **Estado:** Pendientes | Hechas | Todas (default: Pendientes).
- **Expediente:** autocompletar sobre Mis expedientes → deja la vista mostrando solo ese caso (equivale a mirar su ficha, pero en clave calendario).
- **Búsqueda de texto** libre sobre título/descripción/carátula.
- **Vista Lista agrupada por secciones** (la separación pedida — el usuario nunca ve una sopa de entradas mezcladas):
 - ⚠ Vencidas sin confirmar → Hoy → Esta semana → Próximas → Sin fecha (tareas/gestiones) → Notas (aparte, al final: son registro, no pendientes).
 - Dentro de cada sección, orden cronológico y el color/ícono del tipo siempre visible en cada fila.
- **Grilla mensual** con puntos de color por tipo (+ leyenda de colores fija al pie); clic en un día → lista de sus entradas; clic en una entrada → detalle/edición.
- **Panel lateral de tareas** (las sin fecha + las de la semana), con checkboxes — el gesto Google Tasks.
- **" + Nueva entrada":** modal con tipo, título, fecha (con la calculadora de días hábiles al lado: [hoy + [5] días hábiles ▶ 14/07/2026]), expediente (autocompletar sobre Mis expedientes, o vacío = suelta), descripción, repetición, aviso.
- Cada entrada vinculada muestra la carátula como chip clickeable → abre la ficha del caso.

5.2 Sección "Mis expedientes" (nueva, subsección de Bitácora o hermana en el menú)

Listado:

Mis expedientes (8)		[buscar...] [↓ Exportar] [+ Agregar]		
Expediente	Carátula	Situación	Pendien.	Últ.act.
FCR 1234/21	PEREZ c/ GOMEZ s/...	EN DESPACHO	● 2	05/07
CIV 887/23	RUIZ c/ BANCO s/...	A SENTENCIA	—	28/06

Ficha del expediente (al abrir una fila):

FCR 1234/2021 – PEREZ c/ GOMEZ s/ DAÑOS
[← Editar] [🗑]

Juzgado Federal 2 · Comodoro Rivadavia

Situación actual: EN DESPACHO (05/07/2026)

🔔 Próximo vencimiento: 12/08 – Contestar traslado

📅 ENTRADAS DE ESTE CASO (4) [+ Nueva entrada]

☐ 12/08 ● Vencimiento – Contestar traslado
☐ 20/08 ● Audiencia – Testimonial (10:00, Sala 3)
☒ 04/07 ● Tarea – Revisar despacho (hecha)
☐ 03/07 ● Nota – Cliente avisado del estado

📁 HISTORIAL DEL CASO

Procuraciones guardadas: [Última – 05/07/2026 ▼] [👁 Ver]

- Última – 05/07/2026
- Anteuúltima – 28/06/2026

Informes guardados: – No hay informes guardados –

El historial se consulta por modal: el usuario elige en el selector cuál snapshot quiere ver — solo se ofrecen los que existen: "Última — {fecha}" y, si hay una segunda, "Anteuúltima — {fecha}"; si no hay ninguno del tipo, la línea dice "No hay procuraciones/informes guardados" (sin selector ni botón). Al presionar 👁 **Ver** se abre un **modal** con el contenido guardado de ese snapshot: fecha de la corrida, situación registrada en ese momento, la lista de movimientos capturados (y, si el recorte del deep-link omitió movimientos, la leyenda "+N más en el visor local de esa fecha"). El modal es de solo lectura — el historial no se edita, se reemplaza solo con nuevas corridas.

La ficha es la **vista integral del caso**: reúne en una sola pantalla los datos del expediente, las entradas de bitácora vinculadas y el historial de procuraciones/informes. (La sección Bitácora, en cambio, es la vista temporal: el calendario de todo.)

- **Los vencimientos del caso están siempre a la vista:** la ficha muestra una línea destacada "🔔 Próximo vencimiento" en el encabezado (el pendiente con fecha más cercana) y el bloque de Entradas lista primero los pendientes ordenados por fecha (vencimientos y

audiencias arriba), después lo hecho y las notas. La posición del bloque de Entradas dentro de la ficha es configurable (personalización, abajo).

- **Editar:** todos los campos de la ficha (§3.1). Las entradas del caso se editan acá mismo o desde la vista global — es el mismo registro (§3.2).
- **Eliminar seguimiento:** confirmación + elección sobre las entradas vinculadas (conservar como sueltas / eliminar). Borra ficha + snapshots.
- El **historial** es de solo lectura y acotado por diseño (hasta 2+2 por caso); se consulta con el selector última/anteúltima + modal (mockup de arriba).

Personalización de la ficha (para no colapsar la pantalla):

- **Orden de las secciones:** el usuario puede reordenar los bloques de la ficha (Entradas / Historial / Notas) con flechas ▲▼ o arrastre simple — el que vive de los vencimientos pone Entradas arriba; el que la usa como carpeta de consulta pone el Historial primero.
- **Cantidad de registros visibles por sección:** selector por bloque (mostrar 3 / 5 / 10), con default 5. Cada bloque muestra los más recientes hasta ese tope.
- **"Ver todos" en modal:** si hay más registros que el tope, el pie del bloque muestra "Ver todos (N) →" que abre un **modal a pantalla completa** con la lista íntegra, buscador y los mismos checks/acciones — se trabaja ahí sin perder la ficha de fondo.
- La preferencia se guarda **por usuario y aplica a todas sus fichas** (una configuración, no una por caso — evita el laberinto), como JSON de preferencias junto a `home_section`.

5.3 Exportación e importación — backup y restauración del usuario

El usuario es dueño de su información y tiene que poder llevársela. Botón "↓ Exportar" visible en Bitácora y en Mis expedientes, que abre un modal simple:



↓ Exportar mi información

Qué exportar:

- ☒ Todo (expedientes + bitácora)
- ☐ Solo entradas de bitácora [rango 📅]
- ☐ Solo un expediente [elegir ▼]

Formato:

- ☒ Excel — para leer y trabajar
- ☐ JSON — backup completo

[Cancelar] [Descargar]

- **Excel (legible):** un archivo con hojas separadas — *Expedientes* (una fila por caso con todos sus campos), *Entradas* (fecha, tipo, título, estado hecho/pendiente, expediente vinculado), *Historial* (snapshots: caso, tipo de corrida, fecha, situación, movimientos). Es el formato que el abogado abre, imprime o archiva. **Solo lectura:** no sirve para restaurar.
- **JSON (backup completo):** volcado íntegro y fiel de fichas + entradas + snapshots, **restaurable** desde el propio portal (ver abajo). Incluye `backup_version`, fecha de exportación y los identificadores internos de cada registro — eso permite que una re-importación reconozca sus propios datos sin duplicarlos.
- También **por ficha:** en la ficha de un expediente, "↓ Exportar este caso" (Excel de sus entradas + historial) — útil para adjuntar al legajo físico o pasarle el estado a un colega.
- Técnica: endpoint `GET /usuarios/api/bitacora/export` con parámetros de alcance y formato; generación en el momento (los volúmenes son chicos por diseño — tope 2+2 por caso), descarga directa, nada se almacena en el servidor. Rate limit suave para evitar abuso.
- Nota comercial: la exportación **acompaña al gating del plan** — si el plan pierde la Bitácora, el acceso se bloquea pero (recomendación) **la exportación queda disponible** un tiempo razonable: nunca reteniendo los datos del usuario como rehén, que además es coherente con el discurso de confianza del producto.

Importación / restauración desde un backup JSON

Botón "↑ Restaurar backup" junto al de exportar. El usuario sube su archivo JSON y elige el modo:

Restaurar backup

Archivo: backup-bitacora-2026-06-30.json ✓ válido
(exportado el 30/06/2026 · 42 casos · 118 entradas)

Cómo restaurar:

- () Reemplazar todo — queda SÓLO lo del backup
(se elimina lo actual; se descarga un respaldo antes)
- (•) Combinar — el backup pisa los casos coincidentes,
se conserva lo que está en tu Bitácora y no en el backup

[Cancelar] [Ver vista previa →]

Los dos modos:

Modo	Qué hace	Cuándo se usa
Reemplazar todo	Borra el contenido actual de la Bitácora del usuario y carga exactamente lo del backup.	Volver a un estado anterior conocido ("me equivoqué en algo grande, quiero mi Bitácora del mes pasado").
Combinar (default)	Por cada caso del backup: si existe en la base (misma clave jurisdicción+expediente), el backup lo pisa (ficha, entradas y snapshots de ese caso); si no existe, se crea. Los casos que están en la base y no figuran en el backup se conservan intactos .	Recuperar casos borrados por error o mezclar el backup con el trabajo posterior.

Dos salvaguardas propuestas (la mejora sobre el pedido):

- Vista previa obligatoria antes de aplicar** (dry-run): el servidor analiza el archivo y muestra el impacto exacto antes de tocar nada — "Se crearán 3 casos · se sobrescribirán 12 casos (con sus 31 entradas) · se conservarán 27 casos que no están en el backup" (o, en modo reemplazo: "se eliminarán 8 casos y 22 entradas actuales"). El usuario confirma viendo números concretos, no una advertencia genérica. Elimina el 90% de los accidentes.
- Respaldo automático previo**: al confirmar cualquier importación, el sistema **genera y descarga automáticamente un export JSON del estado actual** antes de aplicar los cambios. Resultado: ninguna restauración es irreversible — si el resultado no era lo esperado, se restaura el respaldo automático y se vuelve al estado anterior. Es la misma disciplina de "backup antes de tocar" que el proyecto ya usa en operaciones (restore-db, resets), llevada al usuario final.

Detalles de comportamiento:

- Deduplicación de entradas**: en modo combinar, las entradas del backup que ya existen (mismo identificador interno, porque el backup salió de esta misma cuenta) se actualizan en lugar de duplicarse; las entradas creadas después del backup en casos no pisados no se tocan.
- Entradas sueltas** (sin expediente): en combinar se aplica el mismo criterio por identificador (actualiza las conocidas, crea las nuevas, conserva las locales que no están en el backup).
- Validación del archivo**: `backup_version` + estructura verificada al subir; archivo ajeno (de otra cuenta), corrupto o editado a mano de forma inválida → rechazo con mensaje claro antes de la vista previa. Tope de tamaño razonable.
- Todo o nada**: la importación corre en una transacción — si algo falla a mitad de camino, no queda un estado intermedio.
- Gate de plan: mismo criterio que el resto del módulo (y la restauración respeta los topes estructurales: máx. 2+2 snapshots por caso).

5.4 Ajustes en secciones existentes

- Mi Plan**: píldora "★ Es tu pantalla principal / ☆ Establecer como principal" (§3.4). Si la Bitácora está deshabilitada por plan, la píldora de Bitácora no aparece y `home_section` vuelve a Mi Plan.
- Menú lateral del portal**: ítem "📁 Bitácora" (con sub-ítem o tab interna "Mis expedientes"). Si el plan no la incluye: ítem visible pero con candado + landing de upsell, o directamente oculto — **a decidir comercialmente** (recomendación: visible con candado, es marketing gratis).
- App Electron — botón en el topbar (única aparición)**: la barra superior de la app (`index.html` , `.topbar`) hoy trae en una sola fila el logo, los tabs Procurar/Informe/Monitor/Descargas, y — al margen derecho, después de un spacer — los controles de ventana (minimizar/maximizar/cerrar). El botón "📁 Bitácora" se agrega **ahí**, entre los tabs y el spacer, como un botón propio (no un tab más: los tabs cambian la vista interna de la app, este abre el portal externo vía SSO) → `openPortalSection('bitacora')` . **Vive en un solo lugar** — no se duplica en el sidebar. Fase 3 (opcional): badge rojo con el conteo de vencidos-sin-confirmar, mismo patrón visual que el badge de "novedades" que ya existe en el ítem Monitor del sidebar.

6. Flujos de usuario (end-to-end)

F1 — Capturar desde una procuración (el flujo estrella):

1. El abogado procura (individual o lote) → el visor se abre automáticamente.
2. Ve que en FCR 1234/2021 hay un traslado → clic en  + → " + Vencimiento".
3. Se abre el portal (ya logueado por SSO): modal pre-cargado con expediente, carátula, dependencia, situación y el movimiento como descripción.
4. Usa la calculadora: "hoy + 5 días hábiles" → fecha calculada → Guardar.
5. Resultado: entrada creada + ficha del caso creada/actualizada + snapshot de la procuración sumado al historial (si había 2, se descarta el más viejo). Toast de confirmación con link "Ver en Bitácora".

F1b — Captura en lote desde una procuración batch:

1. Corre un lote de 30 expedientes → visor batch.
2. Filtra/revisa, tilda los 5 que tuvieron despacho relevante → " + Crear entradas... → Vencimiento".
3. El portal abre la pantalla de revisión del lote: 5 filas pre-cargadas; ajusta el título de una, aplica "hoy + 5 días hábiles" a todas → "Guardar todo".
4. Resultado: 5 entradas + 5 fichas creadas/actualizadas en una sola pasada.

F1c — Reencuentro con un caso ya seguido:

1. Procura de nuevo; en el visor, FCR 1234/2021 aparece con  **ficha** (ya estaba guardado).
2. Clic en  → portal → ficha del caso con sus entradas previas y el historial.
3. Si además quiere agendar algo nuevo, usa el  + de siempre: la entrada se **acumula** en la misma ficha y el snapshot nuevo actualiza el historial (si ya había 2 procuraciones, sale la más vieja).

F2 — Informe individual:

1. Genera el informe → PDF + **mini-visor** (nuevo) del expediente.
2. Misma botonera → mismo flujo que F1, con **origen=informe** (el snapshot va al cupo de informes del historial).

F3 — Solo seguir un caso:

1. En cualquier visor →  + → "  Guardar caso" → ficha creada sin entrada. Toast y listo (cero fricción).

F4 — El ritual de entrada (la pantalla principal):

1. El abogado configuró Bitácora como principal → inicia sesión → banner de avisos arriba.
2. Checkea lo hecho ayer directamente en el banner (un clic por ítem) → revisa los próximos 7 días → si necesita ver más atrás, "ver anteriores".
3. Todo sin abrir ningún modal: el 90% de las visitas diarias son este flujo de 30 segundos.

F5 — Gestión del caso:

1. En Mis expedientes abre la ficha → corrige la carátula (el PJN a veces la trae truncada) → agrega una nota.
2. Meses después, el caso termina → "🗑 Eliminar seguimiento" → elige conservar las entradas históricas como sueltas → ficha eliminada.

F6 — Entrada manual (sin visor):

1. Desde el portal (quizás desde el celular), " + Nueva entrada" → audiencia para la semana próxima, vinculada a un caso ya seguido (autocompletar) o suelta.

F7 — Usuario con plan sin Bitácora:

1. Sus visores no muestran la botonera (solo el pie de upsell, si se decide).
2. En el portal, el ítem con candado explica qué es y qué plan la incluye.

F8 — Backup periódico:

1. Fin de mes: en Mis expedientes → "↓ Exportar" → Todo + Excel → descarga un archivo con sus casos, entradas e historial.
2. Una vez por trimestre baja también el JSON completo y lo guarda con sus resguardos del estudio.
3. Si algún día baja de plan o deja el servicio, ya tiene su información afuera — nada quedó cautivo.

F9 — Recuperación desde un backup:

1. Borró por error el seguimiento de dos casos importantes (o quiere volver al estado del mes pasado).
2. "↑ Restaurar backup" → sube su JSON del 30/06 → elige **Combinar** (recuperar lo borrado sin perder el trabajo posterior).
3. La vista previa le muestra: "se crearán 2 casos (los borrados) · se sobrescribirán 12 · se conservarán 27 que no están en el backup" → confirma.

4. Antes de aplicar, el sistema le descarga automáticamente un respaldo del estado actual — si el resultado no lo convence, restaura ese respaldo con **Reemplazar todo** y queda como estaba.
-

7. Modelo de datos propuesto (borrador)

```
-- Ficha del caso seguido
CREATE TABLE expedientes_seguidos (
  id SERIAL PRIMARY KEY,
  user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  expediente VARCHAR(60) NOT NULL, -- ej. "FCR 1234/2021"
  jurisdiccion VARCHAR(100),
  dependencia VARCHAR(200),
  caratula VARCHAR(300),
  situacion_actual VARCHAR(200), -- última situación registrada
  situacion_fecha DATE, -- fecha de la corrida que la registró
  notas TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE (user_id, jurisdiccion, expediente) -- clave de acumulación (no duplica)
);

-- ▲ Corrección 2026-07-19 (bug de esquema que rompería la deduplicación): para que ese
-- UNIQUE funcione, `jurisdiccion` debe ser NOT NULL DEFAULT '' (cambiar la definición de
-- arriba al implementar). En Postgres los NULL son DISTINTOS entre sí dentro de un UNIQUE:
-- con jurisdiccion nullable, dos fichas del mismo caso sin jurisdicción se DUPLICARÍAN y
-- el ON CONFLICT del upsert de capture nunca matchearía. Alternativa equivalente si se
-- prefiere conservar NULL: índice único por expresión
-- CREATE UNIQUE INDEX ux_exp_seguidos ON expedientes_seguidos (user_id, COALESCE(jurisdiccion, ''), expediente);
-- (con esta variante el upsert debe usar ON CONFLICT sobre la expresión, no sobre la constraint).

-- Historial acotado del caso: últimas 2 procuraciones + últimos 2 informes
CREATE TABLE expediente_snapshots (
  id SERIAL PRIMARY KEY,
  expediente_id INTEGER NOT NULL REFERENCES expedientes_seguidos(id) ON DELETE CASCADE,
  kind VARCHAR(15) NOT NULL, -- 'procuracion' | 'informe'
  run_date DATE NOT NULL, -- fecha de la corrida
  situacion VARCHAR(200),
  data JSONB NOT NULL, -- snapshot compacto (movimientos truncados, resumen)
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Al insertar: DELETE del más viejo si ya hay 2 del mismo kind para ese expediente
-- (la base queda acotada por diseño: máx. 4 filas/caso).
-- ▲ Hallazgo H4 (revisión 2026-07-25): "lógica de aplicación en el endpoint" es
-- AMBIGUO y riesgoso tal como estaba escrito - si el INSERT y el DELETE del más viejo
-- son dos statements separados (SELECT para decidir, luego INSERT, luego DELETE),
-- dos capturas simultáneas del mismo caso (doble clic, o un lote que incluye un caso
-- que también se capturó individual en paralelo) pueden dejar 3+ snapshots, rompiendo
-- la invariante "máx. 4 filas/caso" en la que se apoya el dimensionamiento de $10.
-- CORRECTO: el recorte debe ir en la MISMA transacción del insert, con una sola
-- sentencia atómica tipo:
-- DELETE FROM expediente_snapshots WHERE expediente_id=$1 AND kind=$2
-- AND id NOT IN (SELECT id FROM expediente_snapshots WHERE expediente_id=$1
-- AND kind=$2 ORDER BY created_at DESC LIMIT 1)
-- -- (el INSERT nuevo ya corrió antes en la misma transacción; este DELETE dejando
-- -- el más reciente + el nuevo = 2 filas, sin condición de carrera entre requests)

-- Entradas de bitácora
CREATE TABLE bitacora_entries (
  id SERIAL PRIMARY KEY,
  user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  expediente_id INTEGER REFERENCES expedientes_seguidos(id) ON DELETE SET NULL,
  -- SET NULL: al borrar el caso, la entrada queda suelta
  -- (si el usuario elige "eliminar también", la app las borra antes)
  kind VARCHAR(20) NOT NULL, -- 'vencimiento'|'audiencia'|'tarea'|'gestion'|'nota'
  title VARCHAR(300) NOT NULL,
  description TEXT,
  due_at TIMESTAMPTZ, -- NULL = tarea/gestión sin fecha, nota
  all_day BOOLEAN DEFAULT true,
  done_at TIMESTAMPTZ, -- NULL = pendiente · con valor = check de realización
  repeat_rule VARCHAR(20), -- NULL|'weekly'|'monthly'|'yearly'
  meta JSONB, -- campos específicos del tipo (§3.2): lugar/sala,
  -- modalidad+link, carácter, organismo, aviso, etc.
  source VARCHAR(20) DEFAULT 'manual', -- 'manual'|'visor_procuracion'|'visor_informe'
  created_at TIMESTAMPTZ DEFAULT NOW(),
```

```

    updated_at    TIMESTAMPTZ DEFAULT NOW()
);

-- Feriados / inhábiles (calculadora de plazos)
CREATE TABLE feriados (
    id            SERIAL PRIMARY KEY,
    fecha        DATE NOT NULL UNIQUE,
    motivo       VARCHAR(200)
);

-- ⚠ Hallazgo H2 (revisión 2026-07-25): la tabla nace vacía y necesita seed inicial +
-- mantenimiento anual (feriados cambian cada año); Q11 responde "el admin los mantiene"
-- pero ninguna fase de §11 construía un ABM. Se agrega F1.8 (ABM de feriados en el
-- dashboard admin) – ver endpoints abajo y §11/§11.1 corregidos.

-- Gating por plan + preferencia de pantalla principal (columnas additivas)
ALTER TABLE plans ADD COLUMN bitacora_enabled BOOLEAN DEFAULT false;
ALTER TABLE users ADD COLUMN home_section VARCHAR(20) DEFAULT 'plan'; -- 'plan'|'bitacora'
ALTER TABLE users ADD COLUMN bitacora_prefs JSONB; -- personalización: orden de secciones de la ficha,
-- registros visibles por sección, etc.

```

Migraciones 100% additivas. Dimensionamiento: con el tope 2+2 por caso, un usuario intensivo con 200 casos seguidos son ~800 snapshots compactos (JSONB de pocos KB) — despreciable para el VPS actual.

Endpoints (patrón existente del portal, todos con gate de plan salvo aclaración)

⚠ **Reorganizado por fase (revisión 2026-07-25, hallazgo C2).** Los endpoints de **capture** se movieron a **Fase 2** (marcados abajo): se construyen junto a su único consumidor (los visores), en vez de desplegarse a producción sin uso durante toda la Fase 1 — son además el único endpoint anónimo de todo el sistema (§4.1.1), así que minimizar el tiempo que está en producción sin ejercitarse es preferible.

```

— FASE 1 —
GET/POST/PUT/DELETE /usuarios/api/bitacora           - CRUD de entradas (filtros: rango, kind, pendientes, expedien
POST                /usuarios/api/bitacora/:id/done   - check / uncheck de realización
GET                 /usuarios/api/bitacora/avisos     - banner: vencidos sin confirmar (ventana N días) + próximos
GET/POST/PUT/DELETE /usuarios/api/expedientes        - CRUD de fichas (DELETE con flag ?entries=keep|delete)
GET                 /usuarios/api/expedientes/:id     - ficha + entradas + snapshots
GET                 /usuarios/api/bitacora/export     - exportación (params: alcance=todo|entradas|expediente, form
POST                /usuarios/api/bitacora/import     - restauración desde backup JSON (params: modo=reemplazar|com
GET                 /usuarios/api/feriados?year=      - lectura, para date-picker y calculadora (gate de plan)
POST/PUT/DELETE     /admin/feriados                   - ABM de feriados (F1.8, hallazgo H2) – auth admin, NO gate d
PUT                 /usuarios/api/profile (extendido) - home_section

— FASE 2 (junto a los visores, ver hallazgo C2) —
POST                /usuarios/capture                 - recibe el POST-form del visor (body hasta 5MB): stashea el
GET                 /usuarios/api/capture-draft/:id   - la SPA recupera el payload stasheado tras el redirect (Pos
POST                /usuarios/api/expedientes/capture-lote - lote: upsert de fichas (+ snapshots si aplica), tope 200
GET                 /client/bitacora/seguídos        - (app Electron, JWT de app) claves de casos ya seguidos, pa

```

8. Gating por plan — detalle

Punto de control	Comportamiento
<code>plans.bitacora_enabled</code>	Flag por plan, editable desde el form de planes del dashboard admin (checkbox "Incluye Bitácora").
Backend (<code>routes/usuarios.js</code>)	Middleware en todos los endpoints de bitácora/expedientes: 403 con mensaje claro si el plan no la incluye. Es el gate real. ⚠ Excepción explícita (revisión 2026-07-25, hallazgo H5): la versión anterior de este documento decía "middleware en TODOS los endpoints... es el gate real" en esta fila, y en \$5.3/Q6 prometía que <code>GET /usuarios/api/bitacora/export</code> siguiera disponible 90 días después de perder el plan — las dos afirmaciones son incompatibles tal como estaban escritas (un middleware sin excepciones daría 403 inmediato, sin ventana). El middleware de gate debe tener un carve-out explícito para <code>/bitacora/export</code> : en vez de 403 duro, valida <code>bitacora_enabled</code> del plan actual O que hayan pasado menos de 90 días desde que el usuario perdió el flag (columna a agregar, ej. <code>users.bitacora_lost_access_at</code> , seteada por el mismo proceso que cambia de plan). Si no se quiere sostener esa ventana, la alternativa es descartar la promesa de los 90 días y aplicar el 403 duro también a export — pero hay que elegir una de las dos , no dejarlas contradictorias.
Portal	Con flag off: ítem de menú con candado + pantalla explicativa (o oculto, a decidir); píldora "principal" no disponible; <code>home_section</code> forzado a <code>plan</code> .
App Electron / visores	<code>main.js</code> lee el flag de <code>/client/account</code> al generar cada visor e inyecta <code>bitacoraEnabled</code> → botonera presente o ausente. Botón del topbar de la app: mismo criterio (oculto si el plan no la incluye, salvo que se decida mostrarlo con candado como upsell — mismo criterio que el ítem del menú del portal).
Cambio de plan	Al bajar a un plan sin Bitácora los datos no se borran (fichas y entradas quedan en la base); solo se bloquea el acceso. Al volver a subir, todo reaparece. Evita destrucción de datos por decisiones comerciales.
Trial	A decidir: recomendación — Bitácora habilitada durante el trial (es el gancho de conversión más visual del producto), se corta si el plan pago elegido no la incluye.

9. Riesgo y complejidad de la implementación — explicado sin tecnicismos

Para decidir implementar o no: qué se toca, qué no se toca, qué puede salir mal y cómo se contiene.

9.1 Qué se modifica, componente por componente

Componente	Qué se hace	Qué NO se toca	Riesgo
Base de datos (servidor)	Se agregan 4 tablas nuevas y 3 columnas nuevas. Es como sumar cajones nuevos a un mueble: los cajones existentes (usuarios, suscripciones, pagos) no se mueven ni se abren.	Ninguna tabla existente se modifica en su estructura ni en sus datos.	● Muy bajo
Servidor web (backend)	Se agregan los endpoints nuevos de la Bitácora (código nuevo, al costado del existente) + una consulta liviana para la app.	Login, cobro MercadoPago, webhooks, emails, la automatización contra el PJN, la extensión Chrome.	● Bajo — si algo falla, falla la Bitácora; el resto del sistema ni se entera.
Portal web (lo que ve el usuario en el navegador)	Se agregan 2 secciones nuevas (Bitácora y Mis expedientes) + la píldora en Mi Plan. Es la parte con más pantallas nuevas de todo el proyecto.	Las secciones existentes (Mi Plan, Facturación, Soporte, Ayuda) quedan como están.	● Bajo-medio — "medio" por cantidad de horas, no por peligro: un error queda contenido dentro de la sección nueva.
App Electron (la app de escritorio)	Se modifican las plantillas de los visores (checkboxes + botonera 📄), se crea el mini-visor del informe individual, y se agrega un ítem al menú. Requiere publicar una versión nueva de la app.	🚫 Los scripts encriptados NO se tocan (la automatización que corre contra el PJN queda intacta), las credenciales del PJN, el flujo de ejecución, el candado de dispositivo.	● Medio — el riesgo normal de cualquier release de app, con el checklist de siempre (probar con <code>npm start</code> antes de publicar; fix-forward si algo falla).

9.2 Cómo se contiene el riesgo (las redes de seguridad)

- Dos fases independientes:** la Fase 1 (backend + portal) se prueba y publica **sin tocar la app** — si algo no conviene, se ajusta sin haber emitido ningún release de Electron.

- 2. **El flag por plan es un interruptor de apagado:** la Bitácora nace desactivada (`bitacora_enabled=false` en todos los planes). Se enciende para un plan de prueba, se valida con uso real, y recién ahí se abre. Si algo sale mal → se apaga el flag y **ningún usuario la ve**, sin deploy ni rollback.
- 3. **Staging antes de producción:** el flujo ya existente del proyecto (probar en `staging-api` → backup pre-deploy → prod).
- 4. **Migraciones solo-agregar:** como no se modifica nada existente, deshacer la Fase 1 a nivel base es eliminar las tablas nuevas — lo viejo nunca estuvo en riesgo.
- 5. **Backups:** la disciplina actual (backup `.7z` + tag de recupero antes de cada bloque de trabajo) aplica igual.

9.3 Complejidad y esfuerzo (orientativo)

⚠ **Reconciliado con §11.1 (revisión 2026-07-25, hallazgo C5).** La versión anterior de esta tabla y la tabla de §11.1 (agregada el 2026-07-19) no cerraban entre sí: §11.1 le asignaba a un solo sub-bloque (F1.3, el calendario) el mismo presupuesto "4–6 sesiones" que esta tabla le había dado a **toda** la Fase 1 - portal (calendario + ficha + filtros + export + import juntos). Sumando lo que §11.1 detalla sub-bloque por sub-bloque, el total real es mayor. Tabla corregida:

Bloque	Tamaño	Nota
Fase 1 — backend + base (F1.1, F1.2, F1.5, F1.8)	Chico-mediano	Migraciones, CRUD de bitácora/expedientes, gate de plan, píldora+checkbox admin, ABM de feriados. Patrones ya usados en el proyecto. ~2–3 sesiones.
Fase 1 — portal: Bitácora (F1.3)	El sub-bloque más grande	Calendario mes+lista, banner de avisos, modal con calculadora de plazos. UI nueva y densa. 4–6 sesiones (estimación de §11.1, sin cambios).
Fase 1 — portal: Mis expedientes (F1.4)	Mediano	Listado, ficha, edición, borrado con elección sobre entradas. ~1–2 sesiones.
Fase 1 — export + import (F1.6 + F1.7)	Mediano-grande	F1.6 (Sonnet, chico-mediano) es dependencia dura de F1.7 (Opus, alto — hallazgo C4): la salvaguarda de "respaldo automático antes de importar" no puede existir sin exportación funcionando. Import es el único tramo que puede destruir datos reales del usuario — el más lento de razonar bien, no de tipear. ~2–3 sesiones combinadas.
Subtotal Fase 1		~9–14 sesiones
Fase 2 — visores + capture + mini-visor + release	Mediano-grande	Incluye ahora los endpoints de <code>capture</code> (movidos de Fase 1, hallazgo C2) + el post-procesado de <code>main.js</code> para los visores de procuración (hallazgo H1, trabajo nuevo no contemplado en la versión anterior) + el mini-visor de informe + el release. ~4–6 sesiones + 1 release de Electron con su checklist.
Total orientativo		~13–20 sesiones de trabajo , repartibles en semanas, sin bloquear otros pendientes (B3, flecos de QA). Rango más ancho que la estimación original porque incorpora el trabajo real de H1 (post-procesado) y C2 (capture en F2) que antes no estaba contado.

En una frase: es una mejora de riesgo técnico bajo (no toca dinero, credenciales ni automatización; nace apagada por flag) cuyo costo real es tiempo de desarrollo, concentrado en las pantallas nuevas del portal.

10. Costos estimados (infraestructura)

Medición real del servidor al 2026-07-05: disco total 49 GB, usados 5,1 GB → **44 GB libres**; base de datos de producción completa: **13 MB**; droplet DigitalOcean actual de 2 GB RAM.

10.1 Cuánto pesa la Bitácora por usuario

Elemento	Peso estimado	Tope estructural
Ficha de expediente seguido	~1 KB	—
Snapshot (procuración o informe, recortado)	~4 KB	máx. 4 por caso (2+2) → máx. ~17 KB por caso
Entrada de bitácora	~0,5 KB	sin tope, pero son texto corto

Usuario intensivo (escenario cargado a propósito): 300 casos seguidos con historial lleno + 1.000 entradas/año → $300 \times 17 \text{ KB} + 1.000 \times 0,5 \text{ KB} \approx \sim 6 \text{ MB por usuario por año}$. Un usuario típico va a estar muy por debajo (30–80 casos → menos de 2 MB).

10.2 Escenarios de crecimiento

Escenario	Usuarios con Bitácora	Espacio estimado	¿Entra en los 44 GB libres?
Beta actual	50	~0,3 GB	Sobra (menos del 1% del disco libre)
Crecimiento medio	200	~1,2 GB	Sobra
Éxito comercial	500 (todos intensivos — sobreestimado)	~3 GB	Sobra

10.3 Conclusión de costos

- **Costo de infraestructura adicional: USD 0/mes.** El servidor actual absorbe cualquier escenario realista de la Beta y bastante más allá; el tope 2+2 por caso hace que el crecimiento sea lineal y acotado por diseño (no hay "archivo histórico infinito" que se acumule).
- **Backups:** el dump diario crece en proporción (hoy ~1 MB; con cientos de usuarios activos podría llegar a decenas de MB) — el esquema actual (DO Spaces + copias locales) lo absorbe sin cambio de plan.
- **CPU/RAM:** consultas simples e indexadas por usuario; sin procesos pesados ni crons nuevos de limpieza (el tope se mantiene solo). El droplet de 2 GB no se ve exigido por esta mejora.
- **El único escenario de gasto futuro:** si el producto escala a miles de usuarios, un upgrade de droplet (+USD 6–12/mes) — decisión lejana y que llegaría por el crecimiento general del negocio, no por la Bitácora en particular.
- **El costo real de la mejora es tiempo de desarrollo** (\$9.3) + un release de Electron. Sin servicios externos nuevos, sin licencias, sin APIs pagas.

11. Plan de implementación por fases

⚠ **Reescrita 2026-07-25 (hallazgos C1-C4, C6).** La numeración de esta sección ahora mapea **1:1** con los sub-bloques F1.1–F1.8/F2.1–F2.7 de §11.1 (antes el punto 3 de Fase 1 se partía en dos sub-bloques y los puntos 4-5 se juntaban en uno solo, generando confusión entre "el punto 3 del plan" y "F1.3 de la tabla de esfuerzo"). Los endpoints de **capture** se movieron de Fase 1 a Fase 2 (hallazgo C2, ver también §7). El deliverable de Fase 2 ahora aclara que incluye deploy de backend, no solo el release de Electron (hallazgo C3).

Fase 1 — Núcleo (backend + portal, sin release de Electron)

1. **(F1.1)** Migraciones (**4 tablas** — `expedientes_seguidos`, `expediente_snapshots`, `bitacora_entries`, `feriados` — + **3 columnas:** `plans.bitacora_enabled`, `users.home_section`, `users.bitacora_prefs`) + seed de feriados AR 2026/2027.
 2. **(F1.2)** Endpoints CRUD de bitácora/expedientes + avisos + gate de plan (con el carve-out de export, hallazgo H5, §8). (Los endpoints de **capture** YA NO van acá — se movieron a Fase 2, punto 2, hallazgo C2.)
 3. **(F1.3)** Portal: sección Bitácora (banner de avisos con checks, vista mes + lista, panel de tareas, modal de entrada con calculadora de plazos).
 4. **(F1.4)** Portal: sección Mis expedientes (listado, ficha, edición, eliminación con elección sobre entradas).
 5. **(F1.5)** Píldoras "Establecer como principal" en Mi Plan y Bitácora + `home_section` en el login del portal + checkbox "Incluye Bitácora" en el form de planes del admin.
 6. **(F1.6) Exportación** (Excel + JSON, global y por ficha) — el backup del usuario desde el día uno. **Dependencia dura de F1.7** (hallazgo C4): la salvaguarda de importación (§5.3, "respaldo automático antes de aplicar") descarga un export del estado actual, así que F1.6 debe estar terminada y probada **antes** de empezar F1.7, no solo "antes en la numeración".
 7. **(F1.7) Importación/restauración** desde backup JSON (modos reemplazar/combinar, vista previa dry-run, respaldo automático previo, transaccional). Requiere F1.6 completo (ver punto 6).
 8. **(F1.8)** ABM de feriados en el dashboard admin (hallazgo H2) — sin esto, la calculadora de plazos de F1.3 no tiene cómo mantenerse actualizada año a año.
- **Entregable:** módulo completo operable a mano desde el portal (entrada manual, sin captura desde visores todavía), con backup y restauración, gateado por plan. Deployable a staging→prod sin release de Electron. (Si hiciera falta acortar la fase, F1.7 es el único candidato razonable a diferir — nunca F1.6, del que depende.)
 - ⚠ **Nota de producto (hallazgo C6):** al cerrar esta fase, la Bitácora **no tiene el diferencial** que motiva la propuesta (§1/§14: "el dato nace de la automatización que ya corre todos los días") — sin Fase 2 es una agenda manual sin ventaja sobre papel o Google Calendar.

No anunciar ni vender la feature al cerrar la Fase 1; usarla para validación interna con el flag `bitacora_enabled` encendido en un plan de prueba únicamente.

Fase 2 — Captura desde los visores (backend + release de Electron)

Requiere DOS despliegues, no uno (hallazgo C3): un deploy de backend (los endpoints de `capture` + `GET /client/bitacora/seguidos`) y un release de Electron. Planificar ambos.

1. **(F2.1)** Botonera  + (mini-menú) + pie de descubrimiento en los 4 visores. ⚠ **Dos mecanismos distintos** (hallazgo H1, ver §4.4 corregido): en el visor de informe batch se edita `generadorvisor.js` + template (`main.js` controla `DATOS_BATCH` directamente); en los 3 visores de procuración se edita `visorModal_template.html` (la botonera) y además `main.js` debe post-procesar el HTML ya generado por el script encriptado para inyectar los datos por usuario (`bitacoraEnabled` , casos ya seguidos) — sin tocar los scripts encriptados, pero es un paso de implementación adicional respecto de lo que decía la versión anterior de este plan.
 2. **(F2.2)** Backend: endpoints de `capture` (`POST /usuarios/capture` , `GET /usuarios/api/capture-draft/:id` , `POST /usuarios/api/expedientes/capture-lote` con el tope de 200 casos/request del hallazgo H3) + el parser específico de 5MB montado antes del router (§4.1.1) + PRG. **Movido acá desde Fase 1** (hallazgo C2) — se construye junto a su único consumidor.
 3. **(F2.3)** **Selección múltiple** en la tabla de los visores (checkboxes + barra de acciones "Guardar casos" / "Crear entradas...") + pantalla de revisión del lote en el portal, contra el endpoint de F2.2.
 4. **(F2.4)** **Marcado de casos ya seguidos**: endpoint `GET /client/bitacora/seguidos` (backend) + consumido por el post-procesado de F2.1 + link  a la ficha desde fila y modal.
 5. **(F2.5)** **Mini-visor del informe individual** (nuevo, desde `main.js` , sin tocar scripts encriptados — el informe individual no genera visor hoy).
 6. **(F2.6)** Deep-links con SSO cuando el visor se abre desde la app.
 7. **(F2.7)** Botón "  Bitácora" en el **topbar** de la app (una sola aparición, junto a los tabs — no en el sidebar) + actualización del **tour de onboarding** (`onboarding/tour.js`): el paso 2 existente (`target: '.tab-nav'` , "Navegación — tabs principales") se extiende para mencionar el botón nuevo, o se agrega un paso propio inmediatamente después si visualmente queda separado de los tabs — el mecanismo de spotlight multi-elemento (`targets: [ ... ]` , usado hoy para agrupar "Ver tour" + "Asistente IA" en una sola card) permite resolverlo sin duplicar pasos.
- **Entregable:** el circuito completo F1/F1b/F1c/F2/F3 (§6). **Deploy de backend** (F2.2, F2.4) + **un release de Electron** (vX.Y.Z) siguiendo el checklist del proyecto — en ese orden, para que el backend esté listo cuando el release empiece a usarlo.

Fase 3 — Pulido y palancas

1. Badge de pendientes en la app (conteo al abrir).
2. Visor del monitor con captura (si el uso de fases 1-2 lo valida).
3. Sugerencias automáticas a partir de novedades del monitor (bandeja de aceptar/descartar) — el diferencial mayor, pero recién cuando el hábito de uso exista.
4. Tipos de entrada personalizados, export .ics — **solo si hay demanda real**.

Dependencias con el roadmap vigente: no pisa B3 (MP producción) ni los flecos del plan de pruebas (U9.3). La fase 1 es solo backend+portal (deploy estándar, sin release de Electron); la fase 2 requiere deploy de backend y release de app (ver nota arriba).

11.1 Modelo y nivel de esfuerzo por sub-bloque (agregado 2026-07-19, corregido 2026-07-25)

Guía para ejecutar cada tramo. Criterio: **Opus** para diseño de esquema, lógica de negocio del gating/cobro y decisiones que afectan datos del usuario (import/restauración destructiva); **Sonnet** para CRUD mecánico, UI y trabajo repetitivo con patrones ya establecidos. El esfuerzo es orientativo y asume las reglas del proyecto (staging → backup → prod; sin tocar scripts encriptados; migraciones additivas).

⚠ **Correcciones 2026-07-25:** F1.2 ya no incluye `capture` (movido a F2.2, hallazgo C2) — su esfuerzo baja de Mediano a Chico-mediano. F2.1 se separa de "F2.1–F2.6" en una fila propia porque ahora incluye el post-procesado de `main.js` para los visores de procuración (hallazgo H1) — sube de esfuerzo. Se agrega F1.8 (hallazgo H2). La fila "F2.1–F2.6" se abre en F2.1 (propia) + F2.2–F2.6 (grupo restante).

Sub-bloque	Modelo	Esfuerzo	Por qué
F1.1 — Migraciones (4 tablas + 3 columnas) + seed feriados	Opus, medio	Chico	El esquema es la fundación: definirlo bien (claves de acumulación, ON DELETE correctos, tope 2+2 por diseño atómico — hallazgo H4) evita retrabajos caros. Es additivo pero conviene razonarlo con cuidado una sola vez.
F1.2 — Endpoints CRUD de bitácora/expedientes + avisos + gate de plan (con carve-out de export, H5)	Opus, medio	Chico-mediano (bajó: sin capture , movido a F2.2)	El gate de plan es el freno real (403 server-side); el carve-out de exportación (§8) necesita razonarse junto con él para no dejarlo inconsistente.
F1.3 — Portal: Bitácora (calendario mes+lista, banner de avisos, modal con calculadora de plazos)	Sonnet, medio	Grande (el mayor de todo)	UI nueva, laboriosa pero de patrón conocido. El calculador de plazos (feriados) y la vista mes son lo más denso; nada de lógica de negocio riesgosa. 4–6 sesiones.
F1.4 — Portal: Mis expedientes (listado, ficha, edición, borrado con elección sobre entradas)	Sonnet, medio	Mediano	CRUD visual + la decisión de "borrar entradas o dejarlas sueltas" (ya resuelta en el modelo con ON DELETE SET NULL).
F1.5 — Píldora "principal" (home_section) + checkbox "Incluye Bitácora" en admin	Sonnet, bajo	Chico	Dos toggles con patrón ya usado (visibility de planes, settings).
F1.6 — Exportación (Excel + JSON)	Sonnet, bajo-medio	Chico-mediano	Serialización directa; el Excel reusa exceljs (ya en el stack). Debe terminar y probarse antes de F1.7 (dependencia dura, hallazgo C4), no solo antes en la numeración.
F1.7 — Importación/restauración (dry-run + respaldo previo + transaccional)	Opus, alto	Mediano	El único tramo que destruye datos del usuario. La validación de pertenencia/estructura, el dry-run obligatorio, el respaldo automático previo (requiere F1.6 funcionando) y la transacción "todo o nada" exigen el máximo cuidado — un error acá borra la bitácora real de alguien.
F1.8 — ABM de feriados (dashboard admin) (<i>nuevo, hallazgo H2</i>)	Sonnet, bajo	Chico	CRUD simple (fecha + motivo) con patrón ya usado en el admin. Sin esto, F1.3 (calculadora de plazos) queda sin forma de mantenerse año a año.
F2.1 — Botonera en los 4 visores + post-procesado de main.js para procuración (<i>hallazgo H1, esfuerzo corregido</i>)	Sonnet, medio	Mediano (<i>subió: incluye el post-procesado, no solo templates</i>)	Dos mecanismos distintos por visor (§4.4): templates estáticos (bajo riesgo, patrón conocido) + un paso nuevo de post-procesado de HTML en main.js para los 3 visores de procuración, que no estaba contemplado en la estimación original. Sigue sin tocar scripts encriptados.
F2.2 — Backend: endpoints de capture (PRG, tope 200 filas H3, parser 5MB) (<i>movido desde F1.2, hallazgo C2</i>)	Opus, medio	Mediano	Es el único endpoint anónimo de todo el sistema (§4.1.1) — el PRG, el upsert idempotente y las 5 protecciones del borrador-anónimo quieren el mismo cuidado que ya tenían en la F1.2 original.
F2.3–F2.6 — Selección múltiple, marcado de seguidos, mini-visor informe, deep-links SSO	Sonnet, medio	Mediano	Edita plantillas de visores (generador_visor.js + templates HTML) — nunca los scripts encriptados . El mini-visor del informe se genera desde main.js . Patrón conocido; el riesgo es el de cualquier release de Electron.
F2.7 — Botón topbar + actualización del tour (onboarding/tour.js)	Sonnet, bajo	Chico	El tour ya tiene el patrón multi-elemento (targets: []); el paso 2 (target: '.tab-nav') existe y se extiende. (<i>Verificado 2026-07-19: la estructura que la propuesta asume es correcta.</i>)
F2 — Deploy de backend + release Electron (<i>aclarado, hallazgo C3</i>)	Sonnet, medio	—	Deploy de F2.2/F2.4 a staging→prod primero , después el release de Electron siguiendo el checklist del proyecto (probar npm start → bump → tag → npm run release → 5 lugares de versión visible → deploy portal/landing).
F3 — Pulido y palancas (badge, captura del monitor, sugerencias por novedades)	Opus para las sugerencias automáticas · Sonnet para lo demás	Variable	Las "sugerencias a partir de novedades del monitor" son el diferencial con lógica no trivial (matching novedad→entrada) → Opus. El resto (badge, captura del monitor) es mecánico → Sonnet. Solo si el uso real de F1/F2 lo valida.

Regla transversal (crítica para no romper nada): cada sub-bloque se valida en **staging** antes de prod, y el flag **bitacora_enabled** nace en **false** en **todos** los planes → aunque algo salga mal, ningún usuario ve la Bitácora hasta encender el flag en un plan de prueba. La Fase

1 completa se prueba y publica **sin emitir ningún release de Electron**. Ver también §11 "Nota de producto" (hallazgo C6): la Fase 1 sola no debe anunciarse ni venderse — es para validación interna.

12. Riesgos y decisiones de diseño

#	Riesgo / decisión abierta	Mitigación / a decidir
1	Alcance del calendario puede inflar la fase 1 (drag&drop, vista horaria)	v1: vista mes + lista, entradas all-day por defecto, repetición simple. Sin drag&drop.
2	Deep-link sin sesión (visor reabierto días después) → pasa por login	Aceptable; el flujo principal (visor auto-abierto) lleva SSO. Los parámetros del deep-link sobreviven al ciclo de login (patrón <code>pending_goto</code> ya existente en el portal).
3	Datos del caso viajan por POST desde un HTML <code>file://</code>	Solo datos que ya son del usuario, por HTTPS hacia nuestro propio dominio (<code>target</code> fijo, sin CORS por ser navegación, no fetch). Sin credenciales ni tokens en el payload. Sin límite de tamaño artificial (§4.1)
4	Duplicados por variaciones del número de expediente (espacios, formato)	Normalizar la clave (uppercase, colapsar espacios) en el upsert de <code>capture</code> . Si aun así se duplica, el usuario puede borrar la ficha sobrante.
5	Feridos/inhábiles varían por jurisdicción	v1: feriados nacionales + ferias, editables por admin; disclaimer "verificá el plazo" junto a la calculadora. Jurisdicciones por usuario: futuro.
6	¿Ítem de Bitácora visible con candado u oculto en planes sin la feature?	Recomendación: visible con candado (upsell). A confirmar.
7	¿Bitácora en el trial?	Recomendación: sí (gancho de conversión). A confirmar.
8	¿Qué plan la incluye? (¿COMBO sí, EXTENSION no? ¿solo planes futuros PRO+?)	Decisión comercial pura; el flag por plan la deja abierta sin costo técnico.
9	Multi-miembro (estudios con varios usuarios)	Fuera de alcance (el modelo actual es 1 cuenta = 1 abogado); el esquema no lo bloquea (un futuro <code>responsable</code> es una columna más).
10	Restauración destructiva (usuario elige "Reemplazar todo" sin entender el alcance)	Vista previa dry-run obligatoria con números concretos + respaldo automático del estado actual descargado antes de aplicar → toda importación es reversible.
11	Backup JSON editado a mano, corrupto o de otra cuenta	Validación de <code>backup_version</code> + estructura + pertenencia al subir; rechazo con mensaje claro antes de la vista previa. Importación transaccional (todo o nada).

Descartado en esta versión (registrado para el futuro)

- **Integración con Google** (Calendar/Tasks API u OAuth): descartada por ahora — fricción de permisos y verificación de app desproporcionadas para el beneficio. Si a futuro se pide "verlo en el calendario del teléfono", el primer paso sería un feed .ics de solo lectura (sin OAuth), no la API.
- **Emails de recordatorio**: descartados — el aviso vive en el banner de la Bitácora al ingresar.
- **Historial completo de corridas en la nube**: descartado — reemplazado por el snapshot acotado 2+2 por caso, que da el valor de consulta sin acumular datos.

13. Preguntas abiertas — para responder antes o durante la implementación

Ninguna bloquea la evaluación de la propuesta; todas tienen default razonable indicado. Completar "Respuesta:" cuando se decidan.

#	Pregunta	Default sugerido	Respuesta
Q1	¿Qué planes incluyen la Bitácora? (¿COMBO sí / EXTENSION no? ¿Solo los futuros PRO+?)	COMBO la incluye; EXTENSION no	—
Q2	¿Se habilita durante el trial (20 usos)?	Sí — es el gancho de conversión más visual	—
Q3	En planes sin Bitácora: ¿ítem visible con candado (upsell) u oculto?	Visible con candado	—
Q4	¿El visor del monitor también captura?	No en v1; evaluar en Fase 3 con datos de uso	—
Q5	¿Tope de casos seguidos por usuario o por plan (ej. 200)? ¿O sin tope?	Sin tope en v1 (el costo por caso es ínfimo); revisar si aparece abuso	—
Q6	Al bajar a un plan sin Bitácora, ¿cuánto tiempo queda disponible la exportación?	90 días (coherente con la retención de CUIT)	—
Q7	¿El mini-visor del informe individual se abre siempre, o se respeta la config "abrir visor automáticamente"?	Respetar la config existente	—
Q8	¿Distinguir "hecho procesal" vs "extraprocesal" en el check (como Lex-Doctor), o alcanza el check simple + campo carácter opcional?	Check simple + carácter opcional en Vencimiento	—
Q9	Nombres finales de las secciones: ¿"Bitácora" y "Mis expedientes" quedan?	Quedan	—
Q10	¿Subida del snapshot completo por la app (sin el recorte del querystring) como evolución?	Resuelta en v6: el cambio a POST-formulario (\$4.1) transporta el snapshot completo desde el primer momento, sin recorte ni evolución pendiente	—
Q11	Feridos: ¿los mantiene el admin desde el dashboard? ¿Se cargan ferias judiciales por jurisdicción o solo la nacional?	Admin los mantiene; v1 solo nacional + ferias de enero/julio. Implementado como F1.8 (ABM en el dashboard admin, agregado 2026-07-25 — antes esta respuesta no tenía sub-bloque que la construyera)	—
Q12	¿La importación/restauración entra en Fase 1 o se difiere? (§11, nota de la Fase 1)	Entra en Fase 1 (la propuesta la incluye completa)	—

14. Conclusión

El módulo convierte el resultado de cada procuración e informe en material de trabajo organizable con un clic, y le da al abogado el ritual de entrada que estos sistemas necesitan para volverse hábito: entrar → ver vencidos sin confirmar → checkear → ver la semana. El modelo de expediente seguido con historial acotado (2 procuraciones + 2 informes por caso) da la sensación de "carpeta del caso" sin el costo de un archivo histórico en la nube, y el gating por plan lo deja listo como palanca comercial desde el día uno. La fase 1 es autocontenida (backend + portal, sin release de app) y la fase 2 cierra el circuito con los cuatro visores, incluyendo el mini-visor nuevo para el informe individual que unifica la experiencia sin tocar los scripts encriptados.